ASRS–HFACS: 4-Week Build Plan

File-by-file mapping of research paper knowledge to implementation

Parth — B.Tech CSE, NSUT

September 2026

Overview

This plan maps every file in the project to the specific research paper(s) it should draw on, week by week. For each file: the paper(s) to feed Cursor, and the exact knowledge/technique to pull from that paper. Work through this in order — later weeks depend on decisions made in Week 1.

Priority if time runs short: protect Weeks 1–2 (data + baseline). Cut Week 3's SafeAeroBERT / LLM-CoT additions before cutting anything earlier.

Week 1 — Data Foundation & Labeling (Stage 0)

Day 1–2: `data_loading.py`

Papers: Liu, Li, Ng, Han & Feng (2025) — *"Accident investigation via LLMs reasoning: HFACS-guided Chain-of-Thoughts"* (Section 4, GAHFACS dataset) + your own Project Plan document.

What to feed Cursor:

- How GAHFACS was actually built: filtered NTSB records to `far-part = 091`, matched "accident narratives" and "findings" tables via the `ev-id` column, used GPT-3.5 to extract witness statements from the `narr_accp` / `narr_accf` columns, then had two human experts recode findings into HFACS 8.0.
- Implement `load_ntsb()` expecting this same raw structure (`ev-id`, `narr_accp`, `narr_accf`, findings table) if pulling directly from NTSB CAROL / `avall.zip`, rather than guessing column names.
- Implement `standardize_schema()` mapping either raw ASRS or raw NTSB exports into the common schema: `report_id`, `source`, `text`, `label`.

Day 3–5: `keyword_weak_labeler.py` (new file)

Paper: Madeira, Melício, Valério & Santos (2021) — *"Machine Learning and NLP for Prediction of Human Factors in Aviation Incident Reports"* (Section 2.2, Table 1).

What to feed Cursor:

- The consistency criterion, verbatim: for a candidate keyword, sample 15 random documents containing it. If **≥ 12 of 15** agree on one HFACS-ML class, label *all* documents

containing that keyword with that class. Otherwise discard the keyword entirely — never force a label.
- Their Table 1 keyword → label mappings as a seed list (e.g. "disorientation / situational awareness" → Condition of Operator + Error; "non-adherence to procedures" → Violation). Adapt the *strategy*, not their exact ASN-specific tags, to your ASRS text.
- Log every keyword tested, its pass/fail result, and how many documents it labeled — this becomes a results table for the report.

Day 6–7: `weak_labeler_cluster.py` (new file, optional second labeler)

Paper: Macedo, Ramos, Maior, Moura & Lins (2026) — *"Automated identification of pilot failures in aviation accidents using a BERT-based classifier and topic modeling"* (Section 3.2, Fig. 4–5, Table 2).

What to feed Cursor:

- Their 3-step pipeline: (1) BERT embeddings, mean-pooled per document, (2) UMAP dimensionality reduction, (3) HDBSCAN clustering (no predefined cluster count) → class-based TF-IDF to name each cluster with its top 5 words.
- Their hyperparameter sensitivity analysis: vary `min_cluster_size` and `min_samples`, track % of unassigned outliers, and pick the config balancing outlier rate against cluster coherence — replicate this as a small grid search so the choice of HDBSCAN parameters is defensible.
- Output cluster → keyword tables so you can manually assign each cluster to one of the 4 HFACS classes, saved to a JSON config file.

Week 2 — Baseline Pipeline & Cross-Domain Test (Stage 1)

Day 8–9: `preprocessing.py`

Papers: Madeira et al. (2021), Section 2.3 + Shi, Zurada, Cao & Guan (2022), Section 3.1.

What to feed Cursor:

- Madeira's 3-stage pipeline: (1) **cleaning** — remove duplicates, remove terrorism-caused incidents as non-representative of normal human error; (2) **normalization** — translate non-English text, lowercase, strip punctuation; (3) **tokenization** — alphabetic-only parsing (strip digits), remove a standard stopword list *and* a domain-specific list (`summary, probable, cause, accident, contribute, factor, find, conclusion, ...`), then lemmatize, then drop words appearing ≤ 5 times corpus-wide.
- Implement this exact order (clean → normalize → tokenize → dual stopword removal → lemmatize → rare-word pruning) since it's a documented, citable choice.

Day 10–12: `baseline_models.py`

Papers: Madeira et al. (2021), Sections 3.1–3.3 + Shi et al. (2022), Section 3.1.

What to feed Cursor:

- The TF-IDF formula and the Word2Vec / Doc2Vec alternatives from Madeira Section 3. Add a config flag `FEATURE_METHOD = "tfidf" | "word2vec" | "doc2vec"` to swap extractors — mirrors the real ablation these papers ran.

- Shi et al.'s finding: TF-IDF slightly beat Word2Vec on classical ASRS baselines, but Word2Vec + deep models did better. Structure the comparison so this same question can be answered on your data.
- Implement `LogisticRegression` and `LinearSVC` as originally planned — no new paper knowledge needed here beyond the feature-extractor swap.

Day 13–14: `evaluation.py`

Papers: Vempati, Woods & Solano (2023), Table 1 + Madeira et al. (2021), Section 4.2 (Eq. 7–9).

What to feed Cursor:

- Madeira's exact metric set: **Micro-F1** as the primary metric, **Macro-F1** and **Precision** as secondary. Implement all three — accuracy alone is misleading under class imbalance.
- The cross-domain sanity check: after evaluation, print the ASRS-vs-NTSB per-class distribution and compare against Vempati's published ASRS numbers (Perceptual 53%, Decision 32%, Skill-Based 26%; CRM precondition 53%). Run this automatically right before the ASRS → NTSB transfer test, and log it to a results file.

Week 3 — Transformer Models & Extra Baselines (Stage 2)

Day 15–17: `transformer_model.py`

Papers: Andrade & Walsh (2023) — *"SafeAeroBERT"* (Section IV) + your original DistilBERT spec.

What to feed Cursor:

- SafeAeroBERT's setup: further pre-trained on 262,935 ASRS + 161,460 NTSB reports; fine-tuned via **binary relevance** (one dense layer per label, binary cross-entropy per class) rather than a single softmax multi-class head. Add a `BASE_CHECKPOINT` config switch (DistilBERT vs. SafeAeroBERT's public checkpoint) using the same fine-tuning loop.
- Their reported pattern — SafeAeroBERT beat base BERT/SciBERT on weather/procedure classification but *not* on human-factors/aircraft classification. Structure output tables the same way (per-class precision/recall/F1, not just aggregate accuracy) so it's visible where domain pretraining actually helps.

Day 18–19: `llm_cot_baseline.py` (new file)

Paper: Liu, Li, Ng, Han & Feng (2025), Section 3.2–3.3 + Appendix C/D (working code).

What to feed Cursor:

- Their Table 1 question set directly: Q1 — did the mishap individual do/fail to do something? → Q2 — performance/skill error? → Q3 — judgment/decision error? → Q4 — known deviation (only asked if Q2 and Q3 are both "no"). Their Eq. 4 logic for resolving overlapping answers into one label.
- Use their **HFACS-CoT+** variant specifically (not HFACS-CoT): ask each question as a *separate* LLM call rather than all at once — their ablation (Table 4) showed this sequential approach fixes logical inconsistencies and lifts F1/recall substantially over a single combined prompt.

- Use their Appendix code as a structural template for the LLM-calling logic in this environment.

Day 20–21: `model_comparison.py` (new file)

Papers: All three model papers above, combined.

What to feed Cursor:

- One unified comparison function that every model family (TF-IDF+LogReg, TF-IDF+SVM, DistilBERT, SafeAeroBERT, LLM-CoT+) reports into: columns = model | domain (in/cross) | Micro-F1 | Macro-F1 | Precision | Recall. This becomes the master table for the Results section.

Week 4 — Explainability, Tuning, Phase Analysis, Writeup (Stage 3–4)

Day 22–23: `explainability.py`

Paper: Andrade & Walsh (2023), Section IV.D — context only, no new technique.

What to feed Cursor:

- Make sure SHAP runs against whichever transformer is currently active (DistilBERT or SafeAeroBERT via the config flag).
- Keep the "top-weighted words per class" method for LogReg as the simpler comparison point (already in the original plan).

Day 24–25: `hyperparameter_tuning.py` (new file)

Paper: Madeira et al. (2021), Sections 4.4–4.5, Table 5, Figures 7–9.

What to feed Cursor:

- Their random-search ranges as a template (train size 0.2–0.8, learning rate 0.0025–0.25 log-uniform, epochs 1–100 log-uniform, etc.) — adapt ranges to your SVM / TF-IDF / DistilBERT hyperparameters instead of their LS/D2V ones.
- Their fANOVA finding: only 2–3 of 7 hyperparameters actually mattered. Compute and print a similar importance ranking (fANOVA if available, else a variance contribution proxy) after ~30–50 random search trials.
- Their finding that Bayesian optimization gains plateaued/reversed past 4 free variables — worth using as a stopping rule if Bayesian opt is also tried.

Day 26–27: `flight_phase_analysis.py` (Stage 4 bonus)

Paper: Vempati, Woods & Solano (2023), Table 2 + Figures 4–7.

What to feed Cursor:

- Their weighted keyword list as the seed phase/environment keyword set: runway (2.96%), aircraft (1.55%), landing (1.18%), right (1.18%), airport (1.0%), approach (0.96%), left (0.96%), pattern (0.86%), around / ctaf / tower / winds (0.73% each). Extend with standard phase vocabulary: taxi, takeoff, climb, cruise, descent, final, go-around, touchdown.

- Their cross-tabulation style (error type × time-of-day × daylight, error type × contributing factor — Figures 4–7) as the template for a heatmap of error-type × flight-phase.

Day 28–30: Cross-domain discussion & final report

Papers: Vempati, Woods & Solano (2023) Discussion section + Liu et al. (2025) Section 5.4 / 5.6 — for your own writing, not Cursor.

What to use yourself:

- Vempati's core argument for *why* ASRS → NTSB transfer may degrade: voluntary reporters over-report perceptual errors and under-report their own decision/skill failures, relative to NTSB's externally investigated ground truth.
- Liu et al.'s LLM-vs-human-expert comparison (Section 5.6, Fig. 8) as a template if comparing your LLM-CoT baseline's per-class F1 against human-only judgment.

Quick-Reference Checklist

Week	File	Primary Paper(s)
1	`data_loading.py`	Liu et al. (2025) — GAHFACS construction
1	`keyword_weak_labeler.py`	Madeira et al. (2021) — consistency criterion
1	`weak_labeler_cluster.py`	Macedo et al. (2026) — BERT+HDBSCAN clustering
2	`preprocessing.py`	Madeira et al. (2021); Shi et al. (2022)
2	`baseline_models.py`	Madeira et al. (2021); Shi et al. (2022)
2	`evaluation.py`	Vempati et al. (2023); Madeira et al. (2021)
3	`transformer_model.py`	Andrade & Walsh (2023) — SafeAeroBERT
3	`llm_cot_baseline.py`	Liu et al. (2025) — HFACS-CoT+
3	`model_comparison.py`	All three model papers
4	`explainability.py`	Andrade & Walsh (2023) — context
4	`hyperparameter_tuning.py`	Madeira et al. (2021) — fANOVA
4	`flight_phase_analysis.py`	Vempati et al. (2023) — keyword table
4	Final report	Vempati et al. (2023); Liu et al. (2025)